

L05 How a neural network learns

A network is a function with many unknown numbers, fitted to examples the way you would fit a line.

LiDAR scans

finishing Monday: depth sensors

- **Spinning** units sweep 360° with a fixed set of elevation beams. **Solid-state** units have no moving parts, a narrower field of view, and a lower price.
- Each beam traces a ring, so the cloud is dense along a ring and sparse between rings: a 32-beam unit at 20 m has rings about 40 cm apart.
- Points arrive over the sweep, so a moving robot measures one scan from many poses.

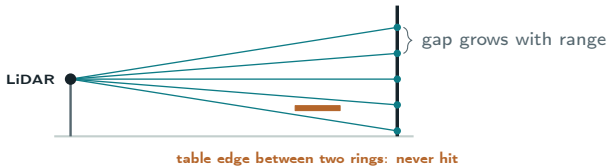


Image: Shvicha, CC BY-SA 4.0, via Wikimedia Commons

Metric versus relative depth

Metric depth is in metres. Relative depth gets the ordering and the shape right but not the size, and a planner or a gripper needs the size.

Source	What you get	Where the metres come from
Stereo	metric	the baseline B , measured once
LiDAR, time of flight	metric	the speed of light and a time
Monocular SfM or SLAM	metric up to one global scale	nowhere: no image fixes it
Monocular depth network	relative: scale and shift unknown per image	nowhere: trained to ignore them
Metric depth network	approximately metric	the intrinsics, or training on one camera

Where learned depth fits

- **Monocular depth networks** such as MiDaS and Depth Anything predict depth from one image, and they are good at *relative* depth.
- Most are trained with a scale- and shift-invariant loss, so the output is correct only up to $aZ + b$, with a and b unknown for each image.
- **Metric** variants take the scale from the intrinsics or from training on one camera; **depth completion** takes it from sparse LiDAR returns. Either way it comes from the sensor or the body, never from the image.

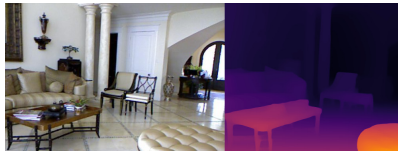
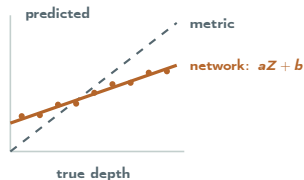


Figure: MiDaS (Intel ISL), MIT License



How a neural network learns

L05 · Unit 0

A depth network is a function with millions of unknown numbers, found from examples the way you would fit a line.

Nothing today assumes you have trained a network before. It starts from $\hat{y} = wx + b$, works one training step by hand, and ends with the encoder–decoder shape of the depth networks on the last slide.

Today

1. **One neuron:** a line with a weight and a bias.
2. **Learning:** guess, check, improve. A loss, gradient descent, and backpropagation, worked by hand.
3. **Layers and activations:** why a stack of lines needs a bend.
4. **Classification:** softmax and cross-entropy.
5. **Training in practice:** batches, epochs, validation, and overfitting.

Learning a function from examples

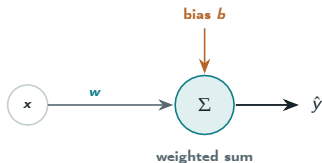
- A **neural network** is a function $\hat{y} = f_{\theta}(x)$ with many unknown **parameters** θ : its weights and biases.
- **Supervised learning**: given pairs (x, y) of inputs and correct answers, choose θ so that $f_{\theta}(x)$ lands near y .
- SIFT and ORB (L04) were functions designed by hand. A network's numbers are found from data.



Image: Josef Steppan, CC BY-SA 4.0, via Wikimedia Commons

You choose the form of the function and the loss that scores a wrong answer; training on data picks the numbers.

One neuron is a line



$$\hat{y} = wx + b$$

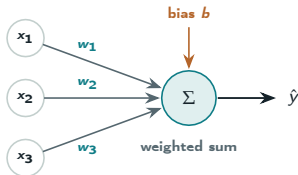
- w is the **weight** and b the **bias**.
- The **forward pass** computes $\hat{y}$ from x with the current w and b .
- Rosenblatt's 1958 **perceptron** was a unit like this, followed by a threshold.



Photo: U.S. Navy, public domain, via Wikimedia Commons

If you knew w and b you would be done. Training is how to find them when you don't.

Many inputs: a weighted sum



$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b = \mathbf{w}^\top \mathbf{x} + b$$

- One weight per input, one bias per neuron.
- The name comes from biology: a neuron sums the signals arriving at its dendrites and fires. The resemblance stops about there.

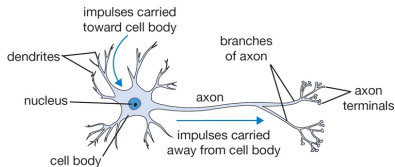
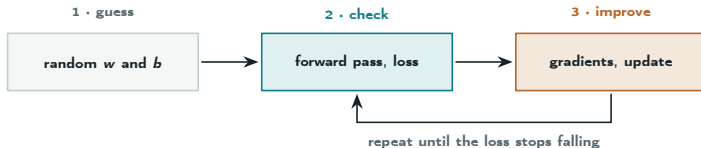


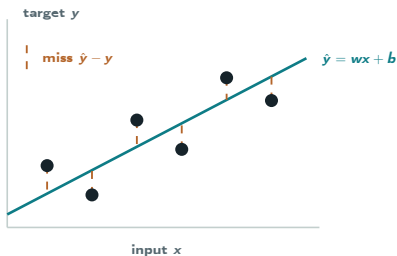
Figure: Stanford CS231n notes, MIT License

Guess, check, improve



- **Training data:** the (x, y) pairs used to fit the weights.
- **Loss:** one number that says how wrong the predictions are.
- **Backpropagation:** work out how much each weight contributed to the loss, then change every weight to reduce it.
- Start from small random weights, drawn from a distribution. A guess is fine; the loop fixes it.

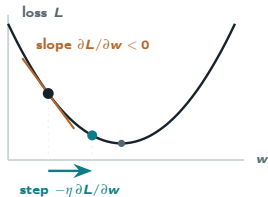
The loss measures the miss



$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 = \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- This is the **mean squared error**: squaring makes a large miss cost far more than a small one.
- The data are fixed, so L is a function of w and b alone. Training means making it small.

Gradient descent: walk downhill



$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

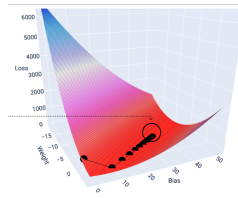
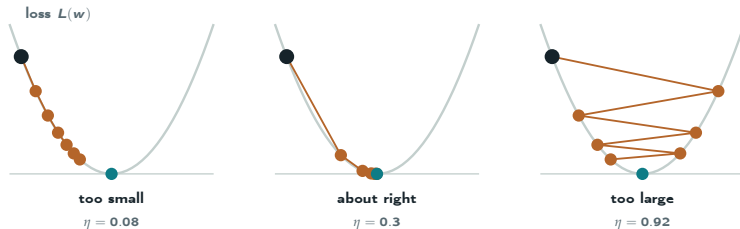


Figure: Google Machine Learning Crash Course, CC BY

4.0

- Positive slope: decrease w . Negative slope: increase it. The **learning rate** η sets the step size.
- Over w and b the loss is a surface (right). With a million weights, each still moves against its own slope.

The learning rate



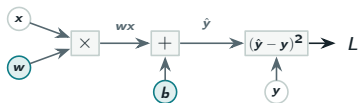
- **Too small:** every step is safe and training takes forever.
- **Too large:** the steps overshoot the bottom and bounce, and noisy data makes it worse. Larger still and the loss diverges.
- Tune it before any other setting. Starting points between 10^{-1} and 10^{-4} are common.

One training step, by hand

1 of 2: forward pass and gradients

One example: $x = 2.1$, target $y = 4$. First guess: $w = 1$, $b = 0$.

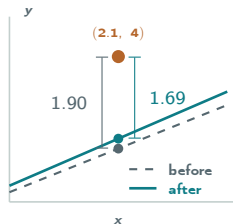
■ The loss, then both gradients by the chain rule



One training step, by hand

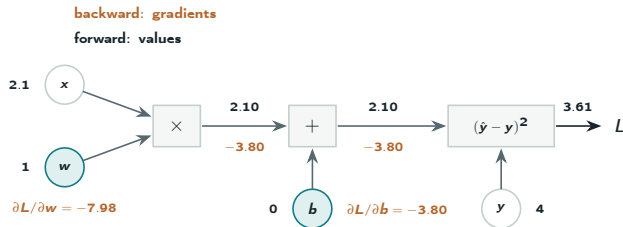
2 of 2: update, and check

■ Take one step with $\eta = 0.01$, then recompute the loss



$\eta = 0.01$ is large for a real network and fine for a toy problem with one example.

Backpropagation is the chain rule on a graph



- **Forward:** compute every node from its inputs, left to right, and keep the values.
- **Backward:** start at the loss and multiply by each node's local derivative on the way back. A node only needs the values stored at its own inputs.
- One backward pass gives the gradient for every weight at once. PyTorch builds this graph as your code runs, and `loss.backward()` walks it.

Stacking lines gives a line

Two neurons in a row, then two side by side, with nothing between them

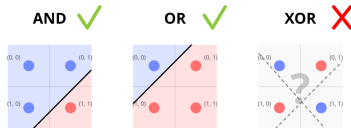
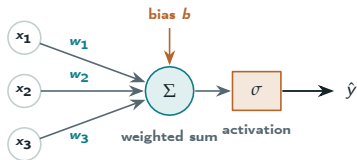


Image: Naokin0525, CC0, via Wikimedia Commons

Any stack of linear layers collapses to a single linear layer, and a linear classifier can only split the data with a straight line. No line separates XOR.

An activation function bends the line

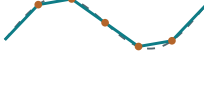


$$\hat{y} = \sigma(w^\top x + b)$$

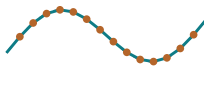
Apply a nonlinear function σ to every neuron's sum, and layers no longer collapse.



one unit
 $\max(0, wx + b)$



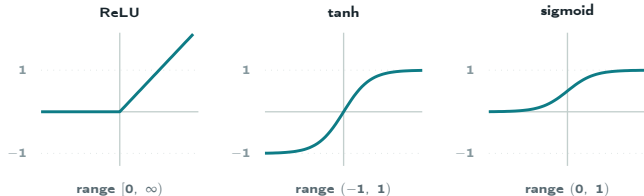
5 units
each orange dot is one bend



14 units
dashed: the function to fit

Each ReLU unit adds one bend. With enough units a single hidden layer can follow almost any continuous function: networks are **universal approximators**.

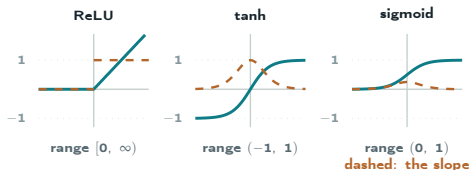
Three common activations



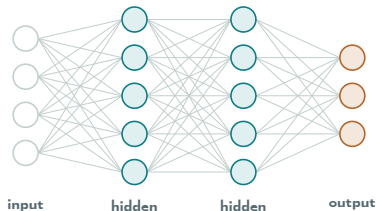
	Formula	Range	Typical use
ReLU	$\max(0, x)$	$[0, \infty)$	hidden layers: the default
tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	$(-1, 1)$	hidden layers in older networks
sigmoid	$1 / (1 + e^{-x})$	$(0, 1)$	the output for a yes-or-no probability

Choosing an activation

- **Range:** bounded or not? A depth output should not be squashed into $(0, 1)$.
- **Differentiability:** backpropagation needs a derivative. ReLU's kink at zero does no harm.
- **Cost:** ReLU is a threshold, and it zeroes negative sums, so fewer units are active.
- **Gradient flow:** the sigmoid's slope is at most 0.25; ReLU's is 1 wherever it is active.
- **The problem:** sigmoid for yes or no, softmax for one of many classes, none for regression.



A multilayer network



$$h_1 = \sigma(W_1x + b_1)$$

$$h_2 = \sigma(W_2h_1 + b_2)$$

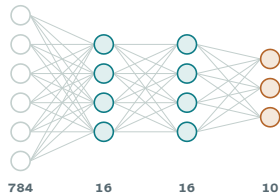
$$\hat{y} = W_3h_2 + b_3$$

- Each layer is a weight matrix, a bias vector and an activation.
- **Fully connected:** every neuron reads every output of the layer before.
- **Hidden** layers are the ones between input and output. **Deep** means many of them.

Counting parameters

A digit classifier: the 28×28 image flattened to 784 inputs, two hidden layers of 16, and 10 outputs, one per digit.

■ Count the weights and biases of a 784–16–16–10 network

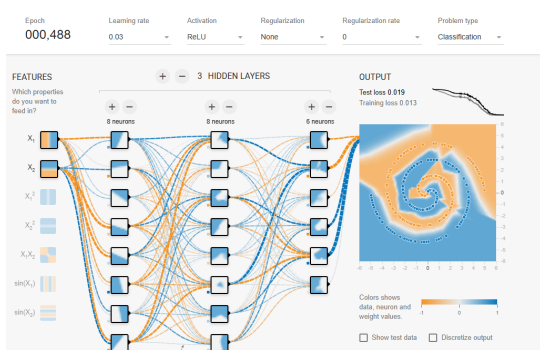


That is the network in 3Blue1Brown's series. Depth Anything's large model has about 335 million.

Watch one learn

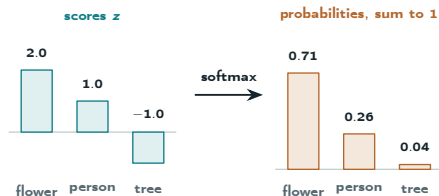
playground.tensorflow.org

1. Pick the spiral dataset. Set the activation to **linear** and train: the boundary stays straight.
2. Switch to **ReLU**. Add a hidden layer and neurons until the spiral separates.
3. Raise the learning rate to 3 and watch the loss jump.



Screenshot: TensorFlow Playground, Apache-2.0

Classification: scores to probabilities



$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- The last layer gives one **score** z_k per class. Scores can be any real number.
- **Softmax** makes them positive and makes them sum to one, and it keeps their order.

Cross-entropy loss

$$L = -\log p_{\text{correct}}$$

■ The loss for the scores on the last slide

- Confident and right costs almost nothing.
Confident and wrong costs a lot.
- Squared error for quantities such as depth;
cross-entropy for categories.

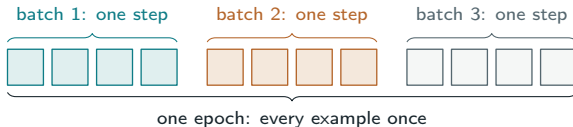


The output layer matches the task

Task	Robot example	Output layer	Loss
Regression	depth at every pixel	none, or ReLU to keep it positive	squared or absolute error
Yes or no	is this pixel free space?	sigmoid	binary cross-entropy
One of many	which of 20 object classes?	softmax	cross-entropy

Semantic segmentation is classification at every pixel, and a depth network is regression at every pixel. The hidden layers can be identical.

Batches and epochs



- A gradient over the whole dataset for every step is too slow. Average it over a random **mini-batch** instead: **stochastic gradient descent**.
- **Epoch**: one complete pass of the training data through the network. 60,000 MNIST images in batches of 100 is 600 steps per epoch.
- Batch size is capped by GPU memory, and 3D data fills memory quickly.

The training loop, in PyTorch

```
model = nn.Sequential(nn.Flatten(),
                      nn.Linear(784, 16), nn.ReLU(),
                      nn.Linear(16, 16), nn.ReLU(),
                      nn.Linear(16, 10))

loss_fn = nn.CrossEntropyLoss()      # softmax, then cross-entropy
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for epoch in range(10):
    for x, y in train_loader:         # one mini-batch at a time
        loss = loss_fn(model(x), y)   # check: forward pass and loss
        opt.zero_grad()               # clear the last step's gradients
        loss.backward()               # backpropagation
        opt.step()                    # improve:  $w = w - lr * \text{gradient}$ 
```

Guess is the random initialisation inside `nn.Linear`. The inner loop is check and improve, once per mini-batch.

Did it learn? Three datasets



- **Training:** used to update the weights.
- **Validation:** never used to update weights, but evaluated periodically during training, to choose the learning rate, the model size, and when to stop.
- **Test:** evaluated once, after every choice is made. It estimates how the model does on data it has never seen.

Trap

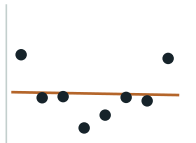
Split robot data by scene or by drive, never by frame. Neighbouring frames are nearly the same image, so a random split puts the test set inside the training set.

Reading the loss curves

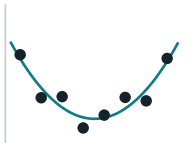


1. Training loss falls, validation loss does not move: the model is fitting the training set and learning nothing that carries over.
2. Both fall: it is learning something about what you are trying to model. Keep going.
3. Training falls while validation rises: **overfitting**. Stop, and keep the checkpoint with the lowest validation loss (**early stopping**).

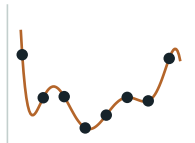
Overfitting



underfit



about right



overfit

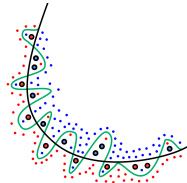


Image: Chabacano, CC BY-SA 4.0,
via Wikimedia Commons

- A model with enough parameters can pass through every training point, noise included, and be wrong everywhere else.
- **Fixes:** more data; **data augmentation** (flips, crops, colour changes); a smaller model; **dropout**; **weight decay**, a penalty on large weights; early stopping.

What you should take away

- A network is a function with many unknown weights, fitted to examples by making a loss small.
- One training step is a forward pass, a loss, the chain rule backwards, and a step against the gradient; you can do it by hand for $\hat{y} = wx + b$.
- Without a nonlinear activation, any stack of layers collapses to one linear layer.
- Softmax turns scores into probabilities, and cross-entropy scores them.
- Validation loss, never training loss, says whether the model will work on data it has not seen.

To go further: 3Blue1Brown, *Neural networks*, chapters 1–4; *Dive into Deep Learning* (d2l.ai), chapters 3–5.

Next time

L06 · The representation catalog, and learning on it

Monday opens with the rest of the deep learning we did not reach: vanishing gradients and normalisation, then convolutional networks, from filters and stride to the encoder–decoder shape of a depth network. Then Unit 1 begins.

- The representation catalog: point clouds, voxels and octrees, signed distance, meshes, implicit surfaces and Gaussians, organised by what the robot has to *do* with them.
- Networks that read point clouds directly: PointNet and PointNet++, then sparse convolution.

Response due Monday: PointNet. **Both diagnostic quizzes are due Sun Sep 27.** Quiz 2's convolution questions come from Monday's first half.